SoilSense: An IoT-Driven Machine Learning Framework for Soil Fertility Classification and Counterfactual Fertilizer Recommendation

Project report in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology

in

CSE (IoT, CS, BT)

Submitted By

Name of the Student: Ruchi Singh

Enrollment Number: 12025002029141

Registration Number: 304202500900308 of 2025-26

Section: B

Under the guidance of

Prof. Subrata Saha

Department of CSE (IoT, CS, BT)



University of Engineering & Management, Kolkata

University Area, Plot No. III – B/5, New Town, Action Area – III, Kolkata – 700160.

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who supported and guided me throughout this project.

First and foremost, I extend my deepest thanks to Prof. Subrata Saha of the Department of CSE (IoT, CS, BT), University of Engineering & Management, Kolkata, for his insightful guidance, encouragement, and patience, which played an essential role in shaping the project's direction. His mentorship has been invaluable, and I feel truly fortunate to have worked under his guidance.

I would also like to express my sincere gratitude to our Head of Department, *Dr. Sandip Mandal* and to all the esteemed faculty members who generously shared their expertise and provided guidance throughout this work. Their encouragement and support have greatly enriched this project.

Lastly, I want to thank my family and friends for their constant encouragement and understanding throughout this journey. Their support has been my anchor, helping me stay motivated and focused.

Signature of the student
Ruchi Singh

CERTIFICATE

This is to certify that the project entitled “SoilSense: An IoT-Driven Machine Learning Framework for Soil Fertility Classification and Counterfactual Fertilizer Recommendation” submitted by Ruchi Singh (Enrollment Number: 12025002029141, Registration Number: 304202500900308 of 2025-26 , Section B), student of the University of Engineering & Management, Kolkata, in partial fulfillment of the requirement for the degree of Bachelor of Technology in Computer Science and Engineering (Internet of Things, Cyber Security & Blockchain Technology), is a bonafide work carried out under the supervision and guidance of Prof. Subrata Saha during the 3rd Semester of the academic session 2025–2026.

The content of this report has not been submitted to any other university or institute. The work presented is fully original, and its quality and performance have been assessed as satisfactory.

Signature of Supervisor **Signature of Head of the Department**

TABLE OF CONTENTS

Section	Title	Page No.
1	Abstract and Keywords	5
2	Introduction and Background	5
3	Problem Definition and Need	6
4	Objectives and Scope	7
5	Literature Review and Existing Solutions	8
6	Proposed Innovation and Novelty	9
7	System Design and Methodology	10
8	Implementation and Prototype Development	16
9	Testing, Results and Discussion	17
10	Impact, Feasibility and Sustainability	20
11	Limitations, Risks and Future Scope	21
12	Conclusion	22
13	References	23
14	Appendices	24
–	Declaration and Approval	30

1. Abstract and Keywords

Abstract

Gradient boosting classified simulated IoT soil data into Low, Medium and High fertility with a macro-F1 of 0.743 on unseen fields, and a counterfactual engine turned 85 % of Low-fertility days into concrete, field-specific advice. Conventional laboratory soil testing is slow, costly and sparse, while IoT probes give continuous but noisy readings. This project, SoilSense, links the two through a five-layer pipeline: an 8-in-1 probe stream (temperature, moisture, EC, salinity, pH, N, P, K), a cleaning stage with range checks and a Hampel filter, five supervised classifiers, a rule-based alert engine and a counterfactual recommendation engine restricted to properties a farmer can control. The pipeline was tested on 115,200 hourly readings from 40 simulated fields over 120 days, generated with noise, calibration drift, spikes and missing data drawn from published field studies. Under grouped five-fold cross-validation, Gradient Boosting (0.743 ± 0.027) and Random Forest (0.742 ± 0.045) clearly outperformed SVM, a decision tree and KNN. Nitrogen, moisture and pH were the most influential features. The cleaning stage reduced fold-to-fold variability by about 40 %, and a corrected version of a published seven-day irrigation and nitrogen rule produced about four alerts of each type per field per season. The full system is a single, reproducible Python program of about 2,000 words suitable for undergraduate teaching and as the starting point for a field trial.

Keywords: soil fertility, Internet of Things, machine learning, gradient boosting, counterfactual recommendation, precision agriculture

2. Introduction and Background

2.1 Background

Soil is the base of almost every food system, yet most farmers in India still learn about their soil from a laboratory report that arrives weeks after the sample was taken. Conventional soil testing involves field sampling, transport, chemical extraction and wet-lab analysis. It is accurate, but it is slow, costly and spatially sparse: one composite sample may stand in for several hectares, and the result describes the field as it was, not as it is. Decisions on fertilizer, irrigation and soil amendments are therefore often made by habit or by blanket recommendations rather than by evidence.

Two technologies have changed what is possible. Low-cost Internet of Things (IoT) sensors can now measure soil temperature, moisture, electrical conductivity (EC), pH and nitrogen–phosphorus–potassium (NPK) levels in place, and send readings to the cloud every few minutes. Machine learning (ML) models can then turn these streams into classifications (is this soil low, medium or high fertility?) and predictions (will moisture fall below a safe level next week?). A recent systematic review of 77 studies published between 2019 and 2024 found that supervised models such as Random Forest, Support Vector Machines and Gradient Boosting give the highest accuracy for soil fertility, pH and nutrient classification on structured data, while Io T sensing improves reliability through continuous measurement [1].

2.2 Motivation

Farmers get soil results weeks late, from sparse lab samples, so fertilizer and irrigation decisions are made by habit. IoT probes give continuous data but are noisy. A bare "low fertility" label also doesn't tell the farmer what to do.

2.3 Project Overview

This project builds **SoilSense**, a reproducible, laptop-scale prototype that links sensing, learning and advice.

3. Problem Definition and Need

3.1 Problem Statement

The same review also exposes the gap this project addresses. Published systems tend to stop at one of three points: they collect sensor data and show it on a dashboard; they train a classifier and report its accuracy; or they issue a recommendation from fixed rules. Few systems join all three, and fewer still tell the farmer *what to change* to move a poor field to a good one. A label such as "low fertility" is not actionable on its own. Selvanarayanan et al. showed that counterfactual recommendations, which describe the smallest change in inputs that would produce a better outcome, can bridge this gap, but their pipeline relies on deep recurrent models and a large private dataset that is hard for a student team to reproduce.

A second, practical problem is data quality. IoT soil probes drift, saturate and drop readings; the review lists sensor reliability, calibration and noise as recurring barriers. Any student system must therefore clean its data before modelling it.

3.2 Evidence of Need

The need for SoilSense rests on three findings from the literature reviewed in this report.

Slow, sparse testing: *Conventional soil testing involves sampling, transport, chemical extraction and wet-lab analysis. It is accurate but slow and costly, and one composite sample may represent several hectares. The result describes the soil as it was on the sampling day, not as it is when the farmer decides on fertilizer or irrigation.*

Partial systems: *Published work tends to stop at one of three points: sensing with a dashboard, classification with an accuracy score or fixed-rule recommendations. Few systems join all three, and the one that adds counterfactual advice depends on deep recurrent models and a large private dataset that a student team cannot reproduce.*

Unreliable sensor data: *The systematic review lists calibration drift, noise and connectivity limits as recurring barriers to IoT soil monitoring, and rates sensor reliability as only moderate. Any practical system must clean its data before modelling it.*

A bare "low fertility" label also does not tell a farmer what to change. Counterfactual advice, which describes the smallest feasible change that would improve the outcome, addresses this directly.

4. Objectives and Scope

4.1 Objectives

This project builds **SoilSense**, a reproducible, laptop-scale prototype that links sensing, learning and advice. Its objectives are to:

1. Simulate a realistic multi-field IoT sensor stream, using value ranges reported in field studies [2], [3], including noise, drift, outliers and missing readings.
2. Clean the stream and aggregate it into daily feature vectors suitable for ML.
3. Compare five supervised classifiers for three-class soil fertility prediction using stratified cross-validation.
4. Explain which soil parameters drive the best model's decisions.
5. Generate two kinds of advice: rule-based irrigation and nitrogen alerts adapted from Shahab et al. [3], and model-based counterfactual recommendations in the spirit of [2] and [10].

4.2 Scope

1. Monitoring eight soil parameters: temperature, moisture, EC, salinity, pH, N, P and K.
2. Cleaning and processing simulated sensor data.
3. Predicting soil fertility using five ML algorithms.
4. Providing field-specific fertilizer and irrigation recommendations.
5. Future integration with real IoT sensors and mobile applications.

4.3 Assumptions and Constraints

Assumptions

1. Simulated soil data represents realistic field conditions.
2. Soil fertility depends on NPK, pH, moisture and EC levels.
3. Machine Learning models can predict soil fertility using sensor data.
4. Sensor readings may contain noise and missing values.

Constraints

1. The project uses simulated data instead of real IoT sensors.
2. Model accuracy depends on data quality.
3. Fertilizer recommendations are indicative, not exact doses.
4. Real-world testing and hardware implementation are not included.

5. Literature Review and Existing Solutions

5.1 Review of Related Work

The literature on ML- and IoT-based soil analysis falls into four strands: sensing systems, fertility and nutrient classifiers, hybrid and deep models, and decision support. This review covers each and closes with the gap SoilSense targets.

5.1.1 IoT sensing for soil monitoring

Shahab et al. [3] built an IoT system around a commercial 8-in-1 probe that reports temperature, moisture, salinity, EC, pH and NPK over RS485, relayed through an ESP8266 Wi-Fi module to a cloud dashboard. In a 1-acre rice field in Rahim Yar Khan, Pakistan, 14 samples showed temperatures of 30.5–33.2 °C, moisture of 60.6–94.1 %, pH of 7.13–8.33, nitrogen of 71–103 mg/kg, phosphorus of 15–19 mg/kg and potassium of 101–141 mg/kg. The authors paired these readings with a seven-day rule for irrigation and nitrogen alerts and fed values into a third-party app for fertilizer doses. The system is practical and cheap, but its decisions are rule-based rather than learned, and its sample size is small.

Selvanarayanan et al. [2] deployed a richer sensor set (moisture, temperature, CO_2, LDR soil colour, NPK, EC, TDS, GPS and a real-time clock) on Arduino and NodeMCU boards in a coffee plantation in Coorg, Karnataka. They monitored land over 20 weeks and published target ranges for coffee soil, such as pH 5.5–6.5, EC 0.5–1.5 mS/cm and soil temperature 20–25 °C. Senapaty et al. [8] similarly combined IoT nutrient sensing with a crop recommendation model, showing that sensing and learning can run on one low-cost stack.

Across these studies the review by Chaudhary et al. [1] notes recurring constraints: sensor calibration drift, environmental noise, power and connectivity limits in rural areas, and low adoption where farmers lack digital training. Its synthesis rates IoT sensor reliability as only moderate and energy efficiency as low to moderate. These findings shaped the noise, drift and missing-data model used in this project's simulator.

5.1.2 Classical ML for fertility and nutrient classification

Supervised ensemble methods dominate soil fertility work on tabular data. Random Forest (RF), introduced by Breiman [9], averages many decorrelated decision trees and handles nonlinear, noisy features well. Islam et al. [4] combined IoT nutrient monitoring with RF, SVM and MLP models for crop recommendation, finding RF among the strongest. The review [1] summarises reported accuracies across studies: roughly 84–92 % for RF, 80–88 % for SVM, 86–94 % for gradient boosting, 70–82 % for k-nearest neighbours and 65–78 % for single decision trees. These ranges come from different datasets and are not directly comparable, but they give a useful prior: ensembles usually beat single trees and distance-based methods on noisy soil data.

5.1.3 Hybrid and deep models

Bhat et al. [5] used gradient-boosted regression trees to tune a deep neural network surrogate for soil suitability classification on 1,148 samples from Sagar Island, West Bengal, reporting near-perfect F1 scores and using explainable AI to rank inputs. Escorcia-Gutierrez et al. [6]

built an ensemble of deep models for nutrient and pH classification. Selvanarayanan et al. [2] used recurrent networks and gated recurrent units on time-series sensor data and reported 94.25 % accuracy, against 81.22 % for RF and 81.66 % for SVM on the same task. Deep models are attractive for sequences and images, but they need larger labelled datasets, more tuning and more compute, and they are harder to explain [1]. For a student project with modest data, a well-tuned tree ensemble is a sensible baseline before any deep model is attempted.

5.1.4 From prediction to recommendation

A classifier answers "what is this soil like?", but a farmer needs "what should I do?". Counterfactual explanations, formalised by Wachter et al. [10], answer this by finding the smallest change to an input that changes a model's decision. Selvanarayanan et al. [2] adapted the idea to agriculture: they compared present and historical sensor states with Manhattan distance and surfaced alternatives close to the current state. Their approach is promising but tightly coupled to deep models and a large private dataset. Rule-based advice, as in [3], is transparent but ignores interactions between parameters, for example that phosphorus availability falls in alkaline soil.

5.2 Existing Solutions and Gap Analysis

The reviewed work leaves three gaps that are realistic for a student team to close:

- **Integration.** Few open prototypes link a noisy sensor stream, a cleaning stage, a validated classifier and actionable advice in one reproducible program.
- **Actionable advice.** Counterfactual recommendation is rarely combined with simple, explainable models and restricted to controllable soil variables.
- **Honest evaluation.** Many papers report a single accuracy figure without cross-validation, class-wise metrics or a statement of data limits [1].

SoilSense addresses all three at prototype scale.

6. Proposed Innovation and Novelty

6.1 Proposed Solution

SoilSense is a five-layer pipeline: probe stream, cleaning, five classifiers, rule-based alerts and a counterfactual engine. It turns noisy readings into a fertility class and field-specific advice. It is one reproducible Python program.

6.2 Innovative Features

The report makes four contributions suited to an undergraduate project. First, it provides a single Python program (Appendix A, about 2,000 words) that runs end to end in under a minute with only NumPy, pandas, scikit-learn and Matplotlib. Second, it documents a corrected version of the published irrigation and fertilizer rule, whose printed pseudocode contains two errors. Third, it implements a simple, transparent counterfactual search that only changes the

soil properties a farmer can actually control. Fourth, it reports results honestly, including where the simulated data limits what can be claimed.

6.3 Novelty Comparison

Shahab et al. [3] use fixed rules only. Islam [4] and Bhat [5] stop at classification. Selvanarayanan [2] gives counterfactuals but needs deep models and a private dataset. SoilSense joins sensing, cleaning, classification, alerts and counterfactuals in a simple program, restricted to controllable soil properties.

6.4 Expected Value

Faster, field-specific advice for farmers. A reproducible teaching base for students. A starting point for a real field trial.

7. System Design and Methodology

7.1 Overall Architecture / Workflow

SoilSense is a five-layer pipeline that runs from the probe in the soil to advice on the farmer's phone (Figure 1). Each layer has one job and passes a clearly defined output to the next, so students can replace any layer, for example swapping the simulator for a real probe, without rewriting the rest.

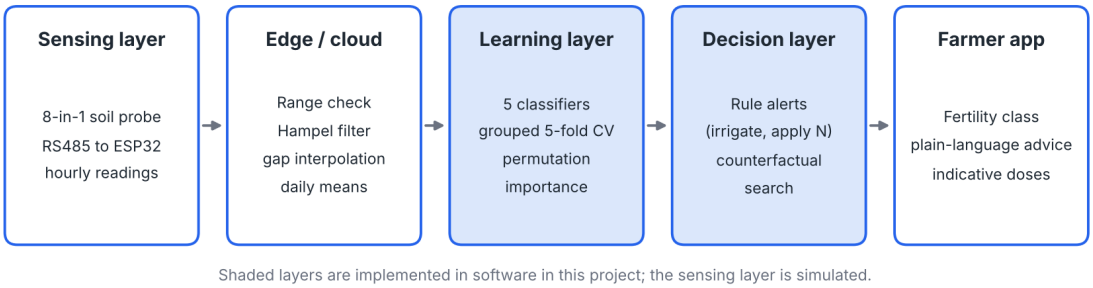


Figure 1. SoilSense architecture. Data flows left to right from the soil probe to the farmer. The shaded learning and decision layers are the software contribution of this project; the sensing layer is simulated with the same eight parameters and hourly cadence as the 8-in-1 probe in [3].

Sensing layer. The reference hardware is the commercial 8-in-1 probe used by Shahab et al. [3]. It reports soil temperature, volumetric moisture, EC, salinity, pH and available N, P and K over an RS485 bus at 9,600 baud, drawing about 15 mA at 12 V. An RS485-to-TTL converter connects it to an ESP32 or ESP8266 microcontroller, which reads the probe once an hour and publishes a JSON message over Wi-Fi or a mobile hotspot using MQTT. Selvanarayanan et al. [2] followed a similar Arduino and NodeMCU design and pushed data to AWS IoT Core. In this project the sensing layer is simulated (Section 7.3), but the simulator emits exactly the eight fields and hourly cadence a real probe would.

Edge / cloud preprocessing layer. Raw readings are never fed straight into a model. This layer rejects values outside the probe's physical range, removes spikes with a Hampel filter, fills

short gaps by interpolation and averages the cleaned hourly values into one record per field per day. Daily aggregation suits soil properties, which change over days rather than minutes, and it cuts the data volume by a factor of 24.

Learning layer. Daily records are classified into Low, Medium or High fertility. Five supervised models are trained and compared under grouped cross-validation, and the best one is explained with permutation importance.

Decision layer. Two engines turn data into advice. A rule engine watches the daily moisture and nitrogen series and raises irrigation and nitrogen alerts. A counterfactual engine takes any field the model calls Low and searches for the smallest realistic change to controllable soil properties that would make the model call it High.

Farmer app layer. The final layer presents the fertility class, active alerts and plain-language advice with indicative doses. A Flutter or web front end, as built in [2] and [3], is outside the scope of this report; the program prints the advice that such an app would display.

The design deliberately keeps learning (what is the soil like?) separate from deciding (what should the farmer do?). Rules are easy to audit and act fast on single parameters; the counterfactual engine captures interactions the model has learned. Running both lets the two kinds of advice be compared and lets an agronomist overrule either.

7.2 Hardware / Software / Tools

The whole system is one Python file, soilsense.py, of about 2,000 words (Appendix A). It ran end to end in 38 seconds on a cloud Linux machine with Python 3.13, NumPy 2.5, pandas 3.0, scikit-learn 1.9 and Matplotlib 3.11; a student laptop should take under two minutes. A fixed seed (42) makes every number in this report reproducible.

7.3 Data Collection and Pre-processing

The experiment uses a simulated dataset of 40 fields monitored hourly for 120 days, giving 115,200 sensor readings and 4,800 daily records. Simulation was chosen for three reasons. Public soil datasets rarely contain hourly IoT streams with field identifiers; the only stream in our sources, the Coorg coffee dataset [2], has different parameters and a different crop. A simulator also lets us know the *true* soil state behind every noisy reading, so we can measure exactly how much damage sensor faults do. Finally, it makes the project fully reproducible from one random seed.

7.3.1 True soil state

Each field receives fixed baselines drawn from ranges that overlap the field values reported in [2] and [3]: nitrogen 45–135 mg/kg, phosphorus 8–30 mg/kg, potassium 80–185 mg/kg, pH 5.4–8.6 and EC 400–2,600 µS/cm. These ranges are deliberately wider than any single study so that all three fertility classes appear. Two properties then evolve day by day:

- **Moisture** loses 0.8–2.2 percentage points a day to evaporation, gains 8–20 points on rain days (12 % chance per day), and is irrigated when it falls below 50 %, with a 50 % chance that the farmer acts on a given day.

- **Nitrogen** declines by 0.1–0.6 mg/kg a day through crop uptake, with top-dressing of 15–30 mg/kg on days 30 and 75 in about 60 % of fields.

Soil temperature follows a slow seasonal wave around 31 °C, close to the 30.5–33.2 °C range observed in [3]. Salinity is generated as 0.9 × EC plus noise, reflecting that the probe derives both from the same conductivity measurement.

7.3.2 Sensor fault model

Every true daily value is expanded into 24 hourly readings, and four kinds of fault are added, following the reliability problems catalogued in the review [1]:

Table 1. Simulated sensor faults

Fault	How it is simulated	Real-world cause
Random noise	Gaussian, e.g. SD 4 mg/kg for N, 0.08 for pH	Electronic noise, soil heterogeneity
Calibration drift	Field-specific linear drift reaching about 1.5 noise SDs by day 120	Electrode ageing, probe fouling
Spikes	1 % of readings shifted by ±15 noise SDs	Loose connectors, power surges
Missing values	3 % of readings dropped	Lost Wi-Fi packets, battery dips

7.3.3 Cleaning pipeline

Cleaning runs per field and per parameter in three steps.

1. **Range check.** Any value outside the probe's datasheet limits (for example pH outside 0–14, or N outside 0–200 mg/kg) is set to missing.
2. **Hampel filter.** For each reading, the median and the median absolute deviation (MAD) of a seven-reading window are computed. A reading more than three scaled MADs from the median is replaced by the median. Unlike a moving average, this removes spikes without blurring genuine step changes such as an irrigation event.
3. **Interpolation and aggregation.** Remaining gaps are filled by linear interpolation within each field, and the 24 hourly values are averaged into one daily record.

Calibration drift is not corrected, because correcting it needs periodic laboratory reference samples, as Shahab et al. did when they cross-checked selected samples at a soil laboratory [3]. Leaving drift in place is a deliberate test of how robust each model is to a fault that real deployments cannot fully remove.

7.3.4 Labels

The label for each daily record comes from the *true* soil state, not the noisy readings, mimicking a laboratory test taken on the same day. Section 7.4.1 defines the index. After applying the thresholds and a 3 % random label flip to represent laboratory and recording error, the class counts are Low 1,361, Medium 1,813 and High 1,626. The classes are reasonably balanced, so accuracy is meaningful, but macro-F1 is still reported because it weights each class equally.

7.4 Algorithm / Design Approach

The method has four parts: a fertility index that defines the target, five classifiers that learn it from sensor data, a rule engine for single-parameter alerts and a counterfactual engine for multi-parameter advice.

7.4.1 Soil fertility index

Each of six parameters is scored between 0 and 1 by a trapezoidal band score. The score is 1 inside the optimal band [lo, hi] and falls linearly to 0 at a distance of half the band width outside it:

$$s(x) = \begin{cases} \max\left(0,\ 1 - \dfrac{\text{lo} - x}{0.5\ (\text{hi} - \text{lo})}\right) & x < \text{lo} \\ 1 & \text{lo} \le x \le \text{hi} \\ \max\left(0,\ 1 - \dfrac{x - \text{hi}}{0.5\ (\text{hi} - \text{lo})}\right) & x > \text{hi} \end{cases}$$

The fertility index is the weighted sum of the six scores:

$$\text{FI} = 0.25\, s_N + 0.20\, s_P + 0.20\, s_K + 0.15\, s_{\text{pH}} + 0.10\, s_{\text{EC}} + 0.10\, s_{\text{moisture}}$$

Table 2. Optimal bands and weights of the fertility index

Parameter	Optimal band	Weight
Nitrogen (mg/kg)	80–140	0.25
Phosphorus (mg/kg)	15–30	0.20
Potassium (mg/kg)	110–190	0.20
pH	6.0–7.5	0.15
EC (µS/cm)	200–1,500	0.10
Moisture (%)	60–85	0.10

Macronutrients carry 65 % of the weight because they dominate fertility assessments in the reviewed studies [1], [5], [6]. pH gets 15 % because it controls nutrient availability; Shahab et al. note that alkaline paddy soils above pH 8 can lock up phosphorus [3]. Days with FI below 0.70 are labelled Low, 0.70–0.85 Medium and 0.85 or above High. The bands are illustrative and crop-agnostic; a real deployment should replace them with the Soil Health Card norms for the crop and district.

7.4.2 Classifiers

Five supervised classifiers from scikit-learn [7] were chosen to span the families compared in the literature [1], [4]:

Table 3. Classifiers compared and their key settings

Model	Key settings	Why included
Decision Tree	max depth 6, balanced class weights	Interpretable baseline
k-Nearest Neighbours	k = 15, standardised inputs	Distance-based baseline
SVM (RBF kernel)	C = 10, balanced class weights, standardised inputs	Strong on small tabular data
Random Forest [9]	300 trees, min 2 samples per leaf, balanced class weights	Most-used ensemble in soil studies
Gradient Boosting	histogram-based, 300 iterations, learning rate 0.05	Best reported family in [1]

Every model receives the same eight cleaned daily features: temperature, moisture, EC, salinity, pH, N, P and K.

7.4.3 Rule-based alert engine

Shahab et al. [3] published a seven-day rule that irrigates after consecutive dry days and applies nitrogen after consecutive days of falling nitrogen. Their printed pseudocode contains two errors that students should notice. The dry-day counter is written as consecutiveDryDays / 1, which never increases; it should be + 1. The nitrogen trigger fires only when N drops by more than 100 mg/kg in one day, which cannot happen when their own readings span 71–103 mg/kg. SoilSense corrects both:

- **Irrigate** when daily moisture stays below 60 % for three consecutive days.
- **Apply nitrogen** when N stays below 80 mg/kg for three consecutive days, or falls more than 15 % below its seven-day maximum, with a 14-day cooldown so that one deficiency does not trigger repeated alerts while applied fertilizer takes effect.

The window shrinks from seven to three days because seven dry days is already a stress event for most crops; the value is a parameter and can be changed.

7.4.4 Counterfactual recommendation engine

A counterfactual answers: *what is the smallest change to this field that would make the model call it High?* Wachter et al. [10] define it as the input closest to the original that receives the desired prediction. Selvanarayanan et al. [2] measured closeness with Manhattan distance between present and historical sensor states. SoilSense combines both ideas with two practical constraints.

Only actionable features may change. Temperature, EC and salinity cannot be adjusted cheaply in a season, so they stay fixed. The search may only:

Table 4. Soil properties the counterfactual search may change

Feature	Allowed change	Farm action
N (mg/kg)	0 to +60	Urea or organic N
P (mg/kg)	0 to +15	DAP or SSP
K (mg/kg)	0 to +60	MOP
pH	−1.5 to +1.0	Gypsum, sulphur or organic matter down; lime up
Moisture (points)	0 to +25	Irrigation

Nutrients can only be added, never removed, which rules out impossible advice such as “reduce nitrogen by 20 mg/kg”.

Cost is normalised Manhattan distance. Each change is divided by that feature's standard deviation so that 10 mg/kg of N and 0.5 pH units are compared on the same scale:

$$\text{cost}(\Delta) = \sum_{k \in A} \frac{|\Delta_k|}{\sigma_k}$$

where A is the set of actionable features. The L1 form favours sparse advice that changes few properties, which is easier for a farmer to act on.

The search is a simple random search, which any undergraduate can follow. For a Low record, 4,000 candidate changes are sampled; each feature changes with probability 0.6, by a uniform amount inside its allowed range. The trained model scores every candidate, and among those with predicted P(High) ≥ 0.6 the cheapest is returned. If none qualifies, the engine reports that no feasible change exists within the limits, which is itself useful: it tells the farmer the problem lies in a property the farm cannot fix quickly, such as salinity.

Finally, nutrient changes are converted into indicative field rates. Assuming a 15 cm plough layer with bulk density 1.3 g/cm³, one hectare holds about 1.95 million kg of soil, so 1 mg/kg corresponds to roughly 1.95 kg/ha of nutrient. This ignores fertilizer use efficiency, fixation and leaching, so the figures are targets for discussion with an agronomist, not doses.

7.5 Module-wise Description

The code is organised so that each function maps to one box in Figure 1:

Table 5. Functions of soilsense.py mapped to the system layers

Function	Layer	What it does
simulate_true_state, simulate_sensor_stream	Sensing	Generate truth and faulty hourly readings
hampel, clean_and_aggregate	Preprocessing	Range check, spike removal, interpolation, daily means
band_score, fertility_index, label_fertility	Labelling	Compute FI and Low / Medium / High labels
get_models, evaluate	Learning	Build five models and run grouped 5-fold CV
rule_alerts	Decision	Corrected irrigation and nitrogen rule
counterfactual, describe	Decision	Cheapest actionable change and plain-language advice
figure_architecture, figure_results	Reporting	Save Figures 1 and 2 as 200 dpi PNG files

8. Implementation and Prototype Development

8.1 Development Process

Three implementation details are worth pointing out to students. First, scaling for KNN and SVM sits inside a scikit-learn Pipeline, so the scaler is fitted only on each training fold and never sees test data. Second, permutation importance is computed on the held-out fold of the best model, not on its training data. Third, an ablation reruns the best model on data aggregated *without* the Hampel filter and interpolation, to measure what the cleaning stage is worth.

8.2 Prototype / Product Details

Running python soilsense.py prints each stage's output and writes four files: the two figures, model_results.csv with fold-averaged metrics, and rule_alerts.csv listing every alert by field and day.

8.3 Integration

To move from simulation to a real farm, only the first stage needs replacing. A student team can log probe readings from an ESP32 to a CSV file with the columns field, day, hour, temp, moisture, ec, salinity, ph, N, P, K, load it in place of simulate_sensor_stream, and attach laboratory fertility labels for a sample of field-days.

8.4 Demonstration Link

The complete source code is given in Appendix A.

9. Testing, Results and Discussion

9.1 Testing Strategy

The most important methodological choice is **grouped** cross-validation. Daily records from one field are highly correlated, because baselines barely move from one day to the next. If records were shuffled into folds at random, the model would see day 40 of field 7 in training and day 41 of the same field in testing, and would score highly by memorising fields rather than learning soil science. StratifiedGroupKFold keeps every field's 120 days in a single fold and balances classes across the five folds, so each test fold contains only unseen fields. This mirrors deployment, where the model must judge a farm it has never seen. Accuracy and macro-F1 are reported as mean ± standard deviation over the five folds.

Feature importance is measured by permutation [7]: each feature is shuffled in the held-out fold ten times, and the average fall in macro-F1 shows how much the model relies on it. Unlike impurity-based importance, this works for any model and is measured on unseen data.

9.2 Results

The two tree ensembles performed best and almost identically: Gradient Boosting reached a macro-F1 of 0.743 and Random Forest 0.742 on fields they had never seen, about 0.08 above SVM and the single tree and 0.17 above KNN (Figure 2a).

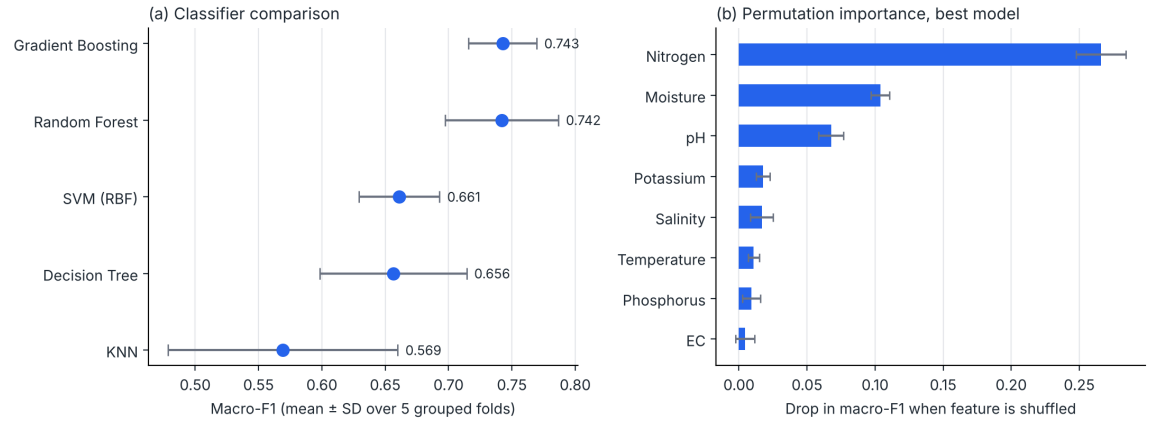


Figure 2. (a) Macro-F1 of five classifiers on unseen fields, mean ± SD over five grouped folds. (b) Permutation importance of the best model (Gradient Boosting) on the held-out fold: the fall in macro-F1 when each feature is shuffled, mean ± SD over 10 repeats.

9.3 Detailed Analysis

9.3.1 Classifier comparison

Table 6. Classifier performance under grouped five-fold cross-validation

Model	Accuracy (mean ± SD)	Macro-F1 (mean ± SD)
Gradient Boosting	0.744 ± 0.021	0.743 ± 0.027
Random Forest	0.745 ± 0.047	0.742 ± 0.045
SVM (RBF)	0.667 ± 0.034	0.661 ± 0.032
Decision Tree	0.672 ± 0.056	0.656 ± 0.058
KNN	0.588 ± 0.092	0.569 ± 0.091

The ranking matches the pattern in the review [1]: boosting and forests lead, single trees and KNN trail. Gradient Boosting is chosen as the final model because its fold-to-fold spread is half that of Random Forest (SD 0.027 against 0.045), meaning its performance depends less on which fields happen to be in the test set. KNN suffers most because distances in eight dimensions are dominated by whichever features are noisiest after scaling, a weakness also noted for noisy IoT data in [1].

The absolute numbers are lower than the 84–94 % accuracies commonly reported [1] and the 94.25 % reported by Selvanarayanan et al. [2]. That gap is expected and informative rather than a failure. The labels contain 3 % random errors, which caps achievable accuracy near 97 %. Uncorrected calibration drift moves each field's readings by up to 1.5 noise SDs by the end of the season. Above all, grouped cross-validation tests only on unseen fields. Many published studies use random splits, which let the model recognise a field from its baseline values and so inflate accuracy. Students should treat any soil-ML paper that does not state how it split fields as optimistic.

9.3.2 Where the errors fall

On the final held-out fold (960 field-days from 8 fields), Gradient Boosting reached 75.9 % accuracy with the following per-class results:

Table 7. Per-class results of Gradient Boosting on the final held-out fold

True class	Precision	Recall	F1	Records
Low	0.738	0.836	0.784	286
Medium	0.730	0.580	0.646	345
High	0.801	0.881	0.839	329

Almost all errors involve the Medium class. Of 345 Medium days, 83 were called Low and 62 High, while only 10 Low days were called High and only 2 High days Low. This is the behaviour one wants from an ordinal problem: the model confuses neighbours, not extremes. It also suggests a design change for real use. Reporting the fertility index as a number with a confidence band, rather than three hard classes, would avoid overstating certainty near the 0.70 and 0.85 cut-offs.

9.3.3 What drives the model

Nitrogen dominates the model's decisions: shuffling it lowers macro-F1 by 0.266, more than twice any other feature (Figure 2b). Moisture (0.104) and pH (0.068) come next; the remaining five features each contribute less than 0.02.

Three observations matter for interpretation. First, permutation importance measures what *this model* relies on *in this data*, not agronomic importance. Nitrogen has the largest weight in the index and is also the only nutrient that changes day to day through uptake and top-dressing, so it explains most changes in class within a field. Phosphorus and potassium carry 40 % of the index weight between them but are fixed per field, so once the model has seen N, moisture and pH they add little. Second, EC shows almost no importance even though it is in the index, because salinity is almost a copy of it: shuffling one leaves the other intact. Correlated features

split their importance, and dropping one of a correlated pair is a reasonable simplification. Third, temperature is not in the index at all yet shows a small positive importance (0.011). Temperature follows the season, and so does nitrogen depletion, so temperature acts as a weak clock. Such spurious signals are exactly why a model trained in one season should be re-validated in another before it is trusted.

9.3.4 Value of the cleaning stage

Removing the Hampel filter and interpolation lowered Gradient Boosting's macro-F1 from 0.743 to 0.728 and raised its fold-to-fold SD from 0.027 to 0.044. The mean gain is modest because averaging 24 hourly readings already dampens most spikes. The larger effect is on stability: cleaning made results 40 % less variable across fields. In a real deployment, where spikes cluster in particular devices, this stability matters more than the average gain.

9.3.5 Rule-based alerts

The corrected rule produced 157 irrigation alerts in 28 of the 40 fields and 176 nitrogen alerts in 24 fields over 120 days, an average of about four of each per field per season. With the original published logic, the irrigation counter would never pass zero and the 100 mg/kg nitrogen trigger would never fire, so the system would have stayed silent throughout. Without the 14-day cooldown added in this project, the same data produced 757 nitrogen alerts, roughly one every six days per field, which farmers would quickly learn to ignore.

9.3.6 Counterfactual recommendations

The counterfactual engine found a feasible improvement for 85 of 100 randomly chosen Low field-days. Five examples show the kind of advice it gives:

Table 8. Example counterfactual recommendations for Low-fertility field-days

Field, day	Suggested change	P(High)	Cost
6, day 1	Raise N by 25 mg/kg (~49 kg/ha N)	0.65	0.86
18, day 110	Lower pH by 0.7 (gypsum, sulphur or organic matter)	0.61	0.80
18, day 119	Raise N by 8 mg/kg; lower pH by 0.5	0.79	0.84
9, day 35	Raise N by 30, P by 6, K by 5 mg/kg	0.83	2.15
29, day 52	No feasible change within limits	–	–

The advice is sparse, as the L1 cost intends: three of four successful cases change only one or two properties. It is also field-specific in a way a fixed rule cannot be. Field 18 is alkaline (true mean pH 8.4), so the engine recommends correcting pH before or alongside adding nitrogen, echoing the phosphorus lock-up concern raised for alkaline paddy soils in [3]. The 15 % of failures are informative too: field 29 is held back by properties outside the farmer's short-term control, here an EC of about 2,190 µS/cm, well above the 1,500 µS/cm band, and needs a longer-term plan such as leaching or gypsum-based reclamation.

Two cautions apply. The engine optimises the *model's* belief, so any error in the model passes straight into the advice. And the kg/ha conversions ignore fertilizer efficiency, which for nitrogen is often well under half, so actual doses would be larger. Counterfactual output should

therefore be read as “which levers matter for this field and roughly how far to move them”, to be confirmed by a soil test and an agronomist.

9.4 Discussion

Gradient Boosting and Random Forest were almost tied (macro-F1 about 0.74) and clearly beat SVM, the single tree and KNN, matching the pattern in the review [1]. Gradient Boosting was chosen because its fold-to-fold spread was half that of Random Forest.

The scores are lower than the 84-94 % in the literature [1], [2]. This is expected: grouped cross-validation tests only on unseen fields, labels carry 3 % noise, and drift was left uncorrected. Random-split studies tend to inflate accuracy.

Errors fell almost entirely on the Medium class, and the model rarely confused Low with High, which is sensible for an ordinal problem. Nitrogen, moisture and pH drove the model, but this reflects the simulated data, not agronomic importance in general.

Cleaning raised the mean macro-F1 only slightly (0.743 to 0.728 without it), but made results about 40 % more stable across fields.

The corrected rule engine gives fast, auditable alerts, while the counterfactual engine found feasible, field-specific advice for 85 of 100 Low field-days. Together they show both quick alerts and which combination of levers matters for a given field.

All results come from simulated data, so they show the pipeline behaves sensibly, not that it would work on a real farm. A one-season field trial with laboratory labels is the necessary next step.

10. Impact, Feasibility and Sustainability

10.1 Expected / Demonstrated Impact

Demonstrated: a macro-F1 of 0.743 on unseen fields, about 40 % more stable results from cleaning, and feasible counterfactual advice for 85 % of Low field-days.

Expected: faster, field-specific advice for farmers, less guesswork in fertilizer and irrigation, and a reproducible base for teaching and a real field trial.

10.2 Social / Environmental Impact

Social. SoilSense aims to give farmers timely, field-specific advice instead of results that arrive weeks late. It can reduce dependence on habit or blanket recommendations, and plain-language advice in local languages can make the information usable for farmers with limited formal education. A low-cost design keeps it within reach of small and marginal farmers. The system supports decisions but does not replace an agronomist, and farmers retain the final choice.

Environmental. Applying fertilizer according to measured need, rather than by blanket rule, can reduce over-application. This lowers nutrient runoff, leaching and cost. The counterfactual engine also avoids impossible advice, since nutrients can only be added within set limits. Irrigation alerts can help avoid both water stress and wasteful watering. Hardware adds some

electronic waste and battery use, which can be reduced by a solar supply and by reusing the probe across seasons. These benefits are plausible but unproven until a field trial measures them.

Scalability And Sustainability

Scalability:

- *Software:* The pipeline is modular, with each layer replaceable (Section 7.1). It processed 40 fields in 38 seconds, and daily aggregation cuts data volume by a factor of 24, so many more fields are feasible.
- *Hardware:* Each field needs one low-cost node, so scaling is mainly a matter of cost and maintenance, not new design.
- *Generalisation:* Moving to a new crop or region requires new fertility bands and fresh labels. A model trained on one region should be re-validated before use in another.

Sustainability:

- All code uses free, open-source libraries and can be maintained by students.
- The fixed seed and single file make the work reproducible and easy to hand over.
- Long-term use needs periodic probe calibration against laboratory samples, plus model re-validation each season.
- Possible models: farmer cooperatives sharing nodes, or agricultural colleges and local agriculture offices supporting maintenance.

11. Limitations, Risks and Future Scope

11.1 Current Limitations

The main limitation is that every result comes from simulated data, so the numbers show how the pipeline behaves, not how well it would work on a real farm.

- **Simulated data.** Value ranges follow published field studies [2], [3], but real soils have spatial structure, correlated nutrients and crop-specific dynamics that the simulator ignores. The model has learned the simulator's rules, including the fertility index that generated its labels.
- **Labels defined by a formula.** The fertility index and its weights are a reasonable, transparent choice, not a validated standard. A different weighting would change the labels and the importance ranking.
- **Drift left uncorrected.** Real probes need periodic laboratory cross-checks, as in [3]; this project shows the cost of skipping them but does not implement a correction.
- **No temporal model.** Each day is classified independently. Recurrent models such as the GRU used in [2] can exploit sequences and forecast future states.

- **Simple counterfactual search.** Random search is easy to understand but neither exhaustive nor guaranteed optimal, and it trusts the model completely.
- **Indicative doses only.** The mg/kg-to-kg/ha conversion ignores fertilizer efficiency, soil type and crop stage.

11.2 Future Enhancements

1. Deploy two or three probes on the campus farm or a partner village plot for one season and send 30–50 samples to a soil testing laboratory for real labels, replacing the simulator.
2. Add a weekly drift correction that re-anchors each probe to its most recent laboratory value.
3. Add a one-dimensional CNN or GRU on 7-day windows to forecast moisture and nitrogen, and compare it with the tree ensembles under the same grouped validation.
4. Replace random search with a constrained optimiser, add per-input cost in rupees, and report several diverse counterfactuals instead of one.
5. Add SHAP explanations for individual predictions, and a simple Flutter front end that shows the class, alerts and advice in Bengali, Hindi and English.
6. Recompute the fertility bands from Soil Health Card norms for one crop, such as kharif paddy in West Bengal.

12. Conclusion

SoilSense shows that a student team can build a complete, honest soil-intelligence pipeline, from noisy IoT readings to field-specific advice, in about 2,000 words of Python. On 4,800 simulated field-days from 40 fields, gradient boosting classified soil fertility into three classes with a macro-F1 of 0.743 on unseen fields, matching Random Forest and clearly ahead of SVM, a single decision tree and KNN. Errors fell almost entirely between neighbouring classes. Nitrogen, moisture and pH drove the model's decisions.

The project's wider lessons matter as much as its scores. Grouped validation gives lower but more trustworthy numbers than the random splits common in the literature. A modest cleaning stage makes results markedly more stable across fields. Published algorithms deserve checking: the irrigation and nitrogen rule reproduced from [3] would never have fired as printed. And a fertility label becomes useful only when it is turned into an action. The counterfactual engine found a feasible, mostly one- or two-step improvement for 85 % of Low field-days, and flagged the remainder as needing longer-term reclamation.

The natural next step is a one-season field trial with real probes and laboratory labels, which would test whether these behaviours hold outside simulation.

13. References

[1] P. Chaudhary, P. Gulia, N. S. Gill, N. Alduaiji, P. K. Shukla and S. Vishnoi, "An evaluation of machine learning for soil analysis in internet of things-enabled smart farming," *Scientific Reports*, vol. 16, art. 10318, 2026. doi:10.1038/s41598-026-36017-z

[2] R. Selvanarayanan, S. Rajendran, S. Algburi, O. I. Khalaf and H. Hamam, "Empowering coffee farming using counterfactual recommendation based RNN driven IoT integrated soil quality command system," *Scientific Reports*, vol. 14, art. 6269, 2024. doi:10.1038/s41598-024-56954-x

[3] H. Shahab, M. Naeem, M. Iqbal, M. Aqeel and S. S. Ullah, "IoT-driven smart agricultural technology for real-time soil and crop optimization," *Smart Agricultural Technology*, vol. 10, art. 100847, 2025. doi:10.1016/j.atech.2025.100847

[4] M. R. Islam, K. Oliullah, M. M. Kabir, M. Alom and M. F. Mridha, "Machine learning enabled IoT system for soil nutrients monitoring and crop recommendation," *Journal of Agriculture and Food Research*, vol. 14, art. 100880, 2023. doi:10.1016/j.jafr.2023.100880

[5] S. A. Bhat, I. Hussain and N.-F. Huang, "Soil suitability classification for crop selection in precision agriculture using GBRT-based hybrid DNN surrogate models," *Ecological Informatics*, vol. 75, art. 102109, 2023. doi:10.1016/j.ecoinf.2023.102109

[6] J. Escorcia-Gutierrez et al., "Intelligent agricultural modelling of soil nutrients and pH classification using ensemble deep learning techniques," *Agriculture*, vol. 12, no. 7, art. 977, 2022. doi:10.3390/agriculture12070977

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] M. K. Senapaty, A. Ray and N. Padhy, "IoT-enabled soil nutrient analysis and crop recommendation model for precision agriculture," *Computers*, vol. 12, no. 3, art. 61, 2023. doi:10.3390/computers12030061

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi:10.1023/A:1010933404324

[10] S. Wachter, B. Mittelstadt and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard Journal of Law & Technology*, vol. 31, no. 2, pp. 841–887, 2018.

14. Appendices

Appendix A: Python source code (soilsense.py)

The complete program, about 2,000 words. Run it with python soilsense.py; it needs numpy, pandas, scikit-learn and matplotlib, and reproduces every number and both figures in this report.

```
"""
SoilSense: IoT-driven ML framework for soil fertility classification
and counterfactual fertilizer recommendation.

Pipeline
  1. Simulate an hourly 8-in-1 IoT soil-probe stream for many fields
     (temperature, moisture, EC, salinity, pH, N, P, K) with realistic
     sensor faults: Gaussian noise, slow calibration drift, spikes and gaps.
  2. Clean the stream (range check -> Hampel filter -> interpolation)
     and aggregate it to one feature vector per field per day.
  3. Label each day Low / Medium / High fertility from a weighted
     agronomic index computed on the TRUE (noise-free) soil state.
  4. Compare five classifiers with grouped, stratified 5-fold CV.
  5. Rank features with permutation importance.
  6. Raise rule-based irrigation / nitrogen alerts (corrected version
     of the seven-day rule in Shahab et al., 2025).
  7. Search counterfactual recommendations: the smallest change to
     controllable soil properties that moves a field to 'High'.

Run:  python soilsense.py          (about 30-60 s on a laptop)
Needs: numpy, pandas, scikit-learn, matplotlib
"""

import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
from sklearn.inspection import permutation_importance

warnings.filterwarnings("ignore", category=FutureWarning)
SEED = 42
rng = np.random.default_rng(SEED)

N_FIELDS, N_DAYS, HOURS = 40, 120, 24
FEATURES = ["temp", "moisture", "ec", "salinity", "ph", "N", "P", "K"]
CLASSES = ["Low", "Medium", "High"]

# Physical limits of the probe (from the 8-in-1 sensor datasheet, Shahab et al.)
LIMITS = {"temp": (-40, 85), "moisture": (0, 100), "ec": (0, 10000),
          "salinity": (0, 10000), "ph": (0, 14), "N": (0, 200),
          "P": (0, 200), "K": (0, 200)}

# Optimal bands used by the fertility index (illustrative; replace with
# Soil Health Card norms for the crop and region being studied).
OPTIMAL = {"N": (80, 140), "P": (15, 30), "K": (110, 190),
           "ph": (6.0, 7.5), "ec": (200, 1500), "moisture": (60, 85)}
WEIGHTS = {"N": 0.25, "P": 0.20, "K": 0.20, "ph": 0.15, "ec": 0.10, "moisture": 0.10}

# Sensor noise (standard deviation per hourly reading)
NOISE = {"temp": 0.5, "moisture": 2.5, "ec": 60, "salinity": 60,
         "ph": 0.08, "N": 4.0, "P": 1.0, "K": 5.0}


# ---------------------------------------------------------------------
# 1. Simulate the IoT stream
# ---------------------------------------------------------------------
def simulate_true_state():
    """Daily true soil state for every field (no sensor error)."""
```

```
    rows = []
    for f in range(N_FIELDS):
        # Field baselines: wide enough to produce all three classes
        base = {"N": rng.uniform(45, 135), "P": rng.uniform(8, 30),
                "K": rng.uniform(80, 185), "ph": rng.uniform(5.4, 8.6),
                "ec": rng.uniform(400, 2600)}
        moisture, n_level = rng.uniform(55, 85), base["N"]
        for d in range(N_DAYS):
            # Moisture: evaporation, random rain, irrigation when very dry
            rain = rng.random() < 0.12
            moisture += -rng.uniform(0.8, 2.2) + (rng.uniform(8, 20) if rain else 0)
            if moisture < 50 and rng.random() < 0.5:
                moisture += rng.uniform(15, 25)            # farmer irrigates
            moisture = float(np.clip(moisture, 35, 98))
            # Nitrogen: slow crop uptake plus occasional top-dressing
            n_level += -rng.uniform(0.1, 0.6)
            if d in (30, 75) and rng.random() < 0.6:
                n_level += rng.uniform(15, 30)
            n_level = float(np.clip(n_level, 20, 190))
            rows.append({
                "field": f, "day": d,
                "temp": 31 + 2 * np.sin(2 * np.pi * d / 120) + rng.normal(0, 0.6),
                "moisture": moisture,
                "ec": base["ec"] + rng.normal(0, 40),
                "ph": base["ph"] + rng.normal(0, 0.05),
                "N": n_level,
                "P": base["P"] + rng.normal(0, 0.5),
                "K": base["K"] + rng.normal(0, 2.0),
            })
    df = pd.DataFrame(rows)
    df["salinity"] = 0.9 * df["ec"] + rng.normal(0, 30, len(df))
    return df


def simulate_sensor_stream(true_daily):
    """Hourly sensor readings = truth + noise + drift + spikes + gaps."""
    hourly = true_daily.loc[true_daily.index.repeat(HOURS)].reset_index(drop=True)
    hourly["hour"] = np.tile(np.arange(HOURS), len(true_daily))
    t = hourly["day"] * HOURS + hourly["hour"]
    for col in FEATURES:
        # Field-specific calibration drift: about 1.5 noise-SDs by season end
        drift_rate = rng.normal(0, NOISE[col] * 1.5 / (N_DAYS * HOURS), N_FIELDS)
        drift = drift_rate[hourly["field"]] * t
        hourly[col] = hourly[col] + rng.normal(0, NOISE[col], len(hourly)) + drift
        spikes = rng.random(len(hourly)) < 0.01           # 1 % spikes
        hourly.loc[spikes, col] += rng.choice([-1, 1], spikes.sum()) * NOISE[col] * 15
        gaps = rng.random(len(hourly)) < 0.03             # 3 % dropped packets
        hourly.loc[gaps, col] = np.nan
    return hourly


# ---------------------------------------------------------------------
# 2. Cleaning and daily aggregation
# ---------------------------------------------------------------------
def hampel(series, window=7, n_sigma=3.0):
    """Replace points far from the rolling median with that median."""
    med = series.rolling(window, center=True, min_periods=1).median()
    mad = (series - med).abs().rolling(window, center=True, min_periods=1).median()
    outlier = (series - med).abs() > n_sigma * 1.4826 * mad
    return series.mask(outlier, med)


def clean_and_aggregate(hourly, use_filter=True):
    df = hourly.copy()
    for col in FEATURES:
        lo, hi = LIMITS[col]
        df.loc[(df[col] < lo) | (df[col] > hi), col] = np.nan     # range check
        if use_filter:
            df[col] = df.groupby("field")[col].transform(hampel)
            df[col] = df.groupby("field")[col].transform(
                lambda s: s.interpolate(limit_direction="both"))
    daily = df.groupby(["field", "day"])[FEATURES].mean().reset_index()
    return daily.dropna()


# ---------------------------------------------------------------------
# 3. Fertility index and labels
# ---------------------------------------------------------------------
def band_score(x, lo, hi):
    """1 inside the optimal band, falling linearly to 0 at 50 % outside it."""
    width = (hi - lo) * 0.5 + 1e-9
    below = np.clip(1 - (lo - x) / width, 0, 1)
```

```
    above = np.clip(1 - (x - hi) / width, 0, 1)
    return np.where(x < lo, below, np.where(x > hi, above, 1.0))


def fertility_index(df):
    return sum(w * band_score(df[k].values, *OPTIMAL[k]) for k, w in WEIGHTS.items())


def label_fertility(index, label_noise=0.03):
    y = np.digitize(index, [0.70, 0.85])                  # 0 Low, 1 Medium, 2 High
    flip = rng.random(len(y)) < label_noise               # lab / annotation error
    y[flip] = rng.integers(0, 3, flip.sum())
    return y


# ---------------------------------------------------------------------
# 4. Model comparison with grouped cross-validation
# ---------------------------------------------------------------------
def get_models():
    return {
        "Decision Tree": DecisionTreeClassifier(max_depth=6, class_weight="balanced",
                                                random_state=SEED),
        "KNN": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=15)),
        "SVM (RBF)": make_pipeline(StandardScaler(), SVC(C=10, gamma="scale",
class_weight="balanced",
                                                         probability=True, random_state=SEED)),
        "Random Forest": RandomForestClassifier(n_estimators=300, min_samples_leaf=2,
class_weight="balanced",
                                                n_jobs=-1, random_state=SEED),
        "Gradient Boosting": HistGradientBoostingClassifier(max_iter=300, learning_rate=0.05,
                                                            random_state=SEED),
    }


def evaluate(X, y, groups, models, n_splits=5):
    """Folds keep every field's days together, so no field leaks into the test fold."""
    cv = StratifiedGroupKFold(n_splits=n_splits, shuffle=True, random_state=SEED)
    results, folds = [], list(cv.split(X, y, groups))
    for name, model in models.items():
        acc, f1 = [], []
        for tr, te in folds:
            model.fit(X.iloc[tr], y[tr])
            pred = model.predict(X.iloc[te])
            acc.append(accuracy_score(y[te], pred))
            f1.append(f1_score(y[te], pred, average="macro"))
        results.append({"model": name, "acc_mean": np.mean(acc), "acc_sd": np.std(acc),
                        "f1_mean": np.mean(f1), "f1_sd": np.std(f1)})
        print(f"  {name:<18} acc {np.mean(acc):.3f} ± {np.std(acc):.3f}   "
              f"macro-F1 {np.mean(f1):.3f} ± {np.std(f1):.3f}")
    return pd.DataFrame(results), folds


# ---------------------------------------------------------------------
# 6. Rule-based alerts (corrected seven-day rule)
# ---------------------------------------------------------------------
def rule_alerts(daily, fc_threshold=60, n_floor=80, n_drop=0.15, k_days=3, n_cooldown=14):
    """Irrigate after k consecutive dry days; fertilize when N stays below
    n_floor for k days or falls more than n_drop over a week.
    Fixes two errors in the published pseudocode: the dry-day counter is
    incremented (+1, not /1) and the N trigger is relative, because an
    absolute 100 mg/kg drop can never occur when readings span 71-103.
    A cooldown stops repeat N alerts while applied fertilizer takes effect."""
    alerts = []
    for f, g in daily.sort_values("day").groupby("field"):
        dry = low_n = 0
        last_n = -n_cooldown
        n_week = g["N"].rolling(7, min_periods=7)
        for (_, row), n_max in zip(g.iterrows(), n_week.max()):
            dry = dry + 1 if row["moisture"] < fc_threshold else 0
            low_n = low_n + 1 if row["N"] < n_floor else 0
            if dry >= k_days:
                alerts.append((f, int(row["day"]), "IRRIGATE"))
                dry = 0
            dropped = pd.notna(n_max) and row["N"] < (1 - n_drop) * n_max
            if (low_n >= k_days or dropped) and row["day"] - last_n >= n_cooldown:
                alerts.append((f, int(row["day"]), "APPLY_N"))
                low_n, last_n = 0, row["day"]
    return pd.DataFrame(alerts, columns=["field", "day", "action"])


# ---------------------------------------------------------------------
# 7. Counterfactual recommendation
```

```
# ---------------------------------------------------------------------
ACTIONABLE = {"N": (0, 60), "P": (0, 15), "K": (0, 60),     # only additions
              "ph": (-1.5, 1.0), "moisture": (0, 25)}        # lime/gypsum, irrigation


def counterfactual(model, x, scale, target=2, p_min=0.6, n_samples=4000):
    """Random search for the cheapest change (normalised Manhattan distance)
    to controllable features that the model predicts as the target class."""
    cand = pd.DataFrame(np.repeat(x.values[None, :], n_samples, 0), columns=x.index)
    deltas = {}
    for k, (lo, hi) in ACTIONABLE.items():
        keep = rng.random(n_samples) < 0.6                # sparse changes
        deltas[k] = np.where(keep, rng.uniform(lo, hi, n_samples), 0.0)
        cand[k] = cand[k] + deltas[k]
    prob = model.predict_proba(cand[x.index])[:, target]
    cost = sum(np.abs(deltas[k]) / scale[k] for k in ACTIONABLE)
    ok = prob >= p_min
    if not ok.any():
        return None
    best = np.argmin(np.where(ok, cost, np.inf))
    return {k: round(float(deltas[k][best]), 2) for k in ACTIONABLE
            if abs(deltas[k][best]) > 1e-6}, float(prob[best]), float(cost[best])


def describe(change):
    """Plain-language advice. 1 mg/kg in a 15 cm layer ~ 1.95 kg/ha (bulk
    density 1.3 g/cm3); treat kg/ha as indicative, not a dose."""
    text = []
    for k, v in change.items():
        if k in ("N", "P", "K"):
            text.append(f"raise available {k} by {v:.0f} mg/kg (~{v*1.95:.0f} kg/ha {k})")
        elif k == "ph":
            text.append(f"{'raise' if v > 0 else 'lower'} pH by {abs(v):.1f} "
                        f"({'lime' if v > 0 else 'gypsum/organic matter'})")
        elif k == "moisture":
            text.append(f"irrigate to raise moisture by {v:.0f} points")
    return "; ".join(text)


# ---------------------------------------------------------------------
# Figures
# ---------------------------------------------------------------------
INK, MUTED, ACCENT, SOFT = "#1f2933", "#6b7280", "#2563eb", "#dbe7fb"


def figure_architecture(path):
    fig, ax = plt.subplots(figsize=(11, 3.2))
    ax.set_xlim(0, 11); ax.set_ylim(0, 3.2); ax.axis("off")
    boxes = [
        ("Sensing layer", "8-in-1 soil probe\nRS485 to ESP32\nhourly readings"),
        ("Edge / cloud", "Range check\nHampel filter\ngap interpolation\ndaily means"),
        ("Learning layer", "5 classifiers\ngrouped 5-fold CV\npermutation\nimportance"),
        ("Decision layer", "Rule alerts\n(irrigate, apply N)\ncounterfactual\nsearch"),
        ("Farmer app", "Fertility class\nplain-language advice\nindicative doses"),
    ]
    w, gap, y0, h = 1.85, 0.32, 0.55, 2.3
    for i, (title, body) in enumerate(boxes):
        x = 0.25 + i * (w + gap)
        ax.add_patch(FancyBboxPatch((x, y0), w, h, boxstyle="round,pad=0.02,rounding_size=0.12",
                                    fc=SOFT if i in (2, 3) else "white", ec=ACCENT, lw=1.4))
        ax.text(x + w / 2, y0 + h - 0.3, title, ha="center", va="center",
                fontsize=10.5, weight="bold", color=INK)
        ax.text(x + w / 2, y0 + h / 2 - 0.2, body, ha="center", va="center",
                fontsize=8.4, color=INK, linespacing=1.5)
        if i < len(boxes) - 1:
            ax.annotate("", xy=(x + w + gap - 0.03, y0 + h / 2), xytext=(x + w + 0.03, y0 + h / 2),
                        arrowprops=dict(arrowstyle="-|>", color=MUTED, lw=1.4))
    ax.text(5.5, 0.18, "Shaded layers are implemented in software in this project; "
            "the sensing layer is simulated.", ha="center", fontsize=8.5, color=MUTED)
    fig.savefig(path, dpi=200, bbox_inches="tight", facecolor="white")
    plt.close(fig)


def figure_results(res, imp, path):
    fig, (a, b) = plt.subplots(1, 2, figsize=(11, 4.2), gridspec_kw={"width_ratios": [1, 1]})
    res = res.sort_values("f1_mean")
    a.errorbar(res["f1_mean"], res["model"], xerr=res["f1_sd"], fmt="o", ms=8,
               color=ACCENT, ecolor=MUTED, capsize=4, lw=1.5)
    for v, sd, m in zip(res["f1_mean"], res["f1_sd"], res["model"]):
        a.text(v + sd + 0.006, m, f"{v:.3f}", va="center", fontsize=9, color=INK)
    a.set_xlabel("Macro-F1 (mean ± SD over 5 grouped folds)", color=INK)
    a.set_title("(a) Classifier comparison", loc="left", fontsize=11, color=INK)
```

```
    names = {"ph": "pH", "ec": "EC", "temp": "Temperature", "moisture": "Moisture",
             "salinity": "Salinity", "N": "Nitrogen", "P": "Phosphorus", "K": "Potassium"}
    imp = imp.sort_values("mean").assign(feature=lambda d: d["feature"].map(names))
    b.barh(imp["feature"], imp["mean"], xerr=imp["sd"], color=ACCENT,
           ecolor=MUTED, height=0.55, capsize=3)
    b.set_xlabel("Drop in macro-F1 when feature is shuffled", color=INK)
    b.set_title("(b) Permutation importance, best model", loc="left", fontsize=11, color=INK)
    for ax in (a, b):
        ax.grid(axis="x", color="#e5e7eb", lw=0.8); ax.set_axisbelow(True)
        for s in ("top", "right"):
            ax.spines[s].set_visible(False)
        ax.tick_params(colors=INK)
    fig.tight_layout()
    fig.savefig(path, dpi=200, bbox_inches="tight", facecolor="white")
    plt.close(fig)


# ---------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------
def main():
    print("1. Simulating sensor stream ...")
    truth = simulate_true_state()
    hourly = simulate_sensor_stream(truth)
    print(f"   {len(hourly):,} hourly readings, "
          f"{hourly[FEATURES].isna().mean().mean():.1%} missing")

    print("2. Cleaning and aggregating ...")
    daily = clean_and_aggregate(hourly).merge(
        truth[["field", "day"]], on=["field", "day"])
    truth_aligned = truth.set_index(["field", "day"]).loc[
        list(zip(daily["field"], daily["day"]))].reset_index()

    print("3. Labelling fertility ...")
    y = label_fertility(fertility_index(truth_aligned))
    print("   class counts:", dict(zip(CLASSES, np.bincount(y, minlength=3))))
    X, groups = daily[FEATURES], daily["field"].values

    print("4. Comparing classifiers (StratifiedGroupKFold, k=5) ...")
    res, folds = evaluate(X, y, groups, get_models())
    best_name = res.sort_values("f1_mean").iloc[-1]["model"]
    print(f"   best model: {best_name}")

    # Ablation: same best model on unfiltered data
    raw_daily = clean_and_aggregate(hourly, use_filter=False)
    raw = raw_daily.merge(daily[["field", "day"]], on=["field", "day"])
    keep = daily.set_index(["field", "day"]).index.isin(raw.set_index(["field", "day"]).index)
    abl, _ = evaluate(raw[FEATURES], y[keep], raw["field"].values,
                      {best_name + " (no filter)": get_models()[best_name]})

    # Detailed report on the last fold
    best = get_models()[best_name]
    tr, te = folds[-1]
    best.fit(X.iloc[tr], y[tr])
    pred = best.predict(X.iloc[te])
    print(classification_report(y[te], pred, target_names=CLASSES, digits=3))
    print("   confusion matrix:\n", confusion_matrix(y[te], pred))

    print("5. Permutation importance ...")
    pi = permutation_importance(best, X.iloc[te], y[te], scoring="f1_macro",
                                n_repeats=10, random_state=SEED, n_jobs=-1)
    imp = pd.DataFrame({"feature": FEATURES, "mean": pi.importances_mean,
                        "sd": pi.importances_std})
    print(imp.sort_values("mean", ascending=False).to_string(index=False))

    print("6. Rule-based alerts ...")
    alerts = rule_alerts(daily)
    print(alerts["action"].value_counts().to_string())

    print("7. Counterfactual recommendations ...")
    best.fit(X, y)                                      # final model on all data
    scale = X.std()
    low_idx = np.where(best.predict(X) == 0)[0]
    examples = rng.choice(low_idx, size=min(5, len(low_idx)), replace=False)
    found = 0
    for i in examples:
        out = counterfactual(best, X.iloc[i], scale)
        tag = f"field {daily['field'].iat[i]:>2}, day {daily['day'].iat[i]:>3}"
        if out is None:
            print(f"   {tag}: no feasible change within limits")
            continue
        found += 1
```

```
            change, p, cost = out
            print(f"   {tag}: P(High)={p:.2f}, cost={cost:.2f} -> {describe(change)}")
        success = np.mean([counterfactual(best, X.iloc[i], scale) is not None
                           for i in rng.choice(low_idx, size=min(100, len(low_idx)), replace=False)])
        print(f"   feasible counterfactuals for 100 Low days: {success:.0%}")

        figure_architecture("fig1_architecture.png")
        figure_results(res, imp, "fig2_results.png")
        res.to_csv("model_results.csv", index=False)
        alerts.to_csv("rule_alerts.csv", index=False)
        print("Saved fig1_architecture.png, fig2_results.png, model_results.csv, rule_alerts.csv")


if __name__ == "__main__":
    main()
```

Declaration and Approval

Student Declaration

We hereby declare that the project titled "SoilSense: An IoT-Driven Machine Learning Framework for Soil Fertility Classification and Counterfactual Fertilizer Recommendation" is the work carried out by the undersigned under the guidance of Prof. Subrata Saha. The information presented in this report is accurate to the best of our knowledge. All external sources, tools, datasets and prior work have been appropriately acknowledged. Any third-party material has been used with due permission or under applicable terms. We understand that project claims and results may be verified by the evaluation committee.

Name: Ruchi Singh Signature: _____ Date: _____

Guide / Mentor Recommendation

Comments: _____

_____

Name: Prof. Subrata Saha Signature: _____ Date: _____

For Department / IEDC / Evaluation Committee Use

Project received on	
Report completeness	☐ Complete ☐ Incomplete
Prototype / Demo verified	☐ Yes ☐ No ☐ Not applicable
Evaluation status	☐ Accepted ☐ Revision required ☐ Not accepted
Evaluator remarks	
Evaluator name & signature	
Date	